

CS 240 - Lab End-Semester Examination

TAs: Sujan, Rishi, Abhi & Malay

Spring 2025

Instructions

- This paper has 4 questions. Attempt all questions.
- Duration: **180 minutes**. Closed book, no internet.
- Create a folder on your Desktop with the following structure. Include only the files listed:

```
submission_<roll_number>/
|-- q1/
|   |-- student_svm.py
|-- q2/
|   |-- student_model.py
|   |-- student_optimizer.py
|   |-- student_train.py
|-- q3/
|   |-- prune.py
|   |-- alpha_bar.pt
|   |-- pruned_model.pth
|-- q4/
|   |-- student_loss.py
```

- Once done, open a terminal, run `check-cs240`, and call a TA. The TA will run `submit-cs240`. **It is your responsibility to ensure your work is submitted.**

Contents

Q1	Efficient SVM Classification [15 marks]	3
Q2	Learning a Nonlinear Function with a FFNN [25 marks]	5
Q3	Seeing What to Lose [30 marks]	8
Q4	Optimising with a Hidden Loss [30 marks]	11
A	Notes on SVM Kernels and Regularisation for Q1	14
B	Note on Optimizers for Q2	15
C	Note on PyTorch Hooks for Q3	17
D	Notes on Nadaraya-Watson and Gradient Computation for Q5	18

Q1. Efficient SVM Classification [15 marks]

You are given a synthetic two-dimensional binary classification dataset. The data is *not* linearly separable, so a suitable nonlinear kernel must be chosen.

Your task is to select a kernel, tune the regularisation parameter C and any kernel-specific parameters so that the resulting SVM classifier meets a minimum accuracy threshold on a held-out test set while using as **few support vectors** as possible.

Background

Given labelled training data $\{(\mathbf{x}_i, y_i)\}_{i=1}^n$ with $\mathbf{x}_i \in \mathbb{R}^2$ and $y_i \in \{0, 1\}$, a Support Vector Machine solves

$$\min_{\mathbf{w}, b, \xi} \frac{1}{2} \|\mathbf{w}\|^2 + C \sum_{i=1}^n \xi_i \quad \text{s.t.} \quad y_i (\langle \mathbf{w}, \phi(\mathbf{x}_i) \rangle + b) \geq 1 - \xi_i, \quad \xi_i \geq 0 \quad \forall i,$$

where ϕ is the feature map induced by the chosen kernel $K(\mathbf{x}, \mathbf{x}') = \langle \phi(\mathbf{x}), \phi(\mathbf{x}') \rangle$.

The *number of support vectors* is a useful proxy for model complexity. Two classifiers with comparable accuracy differ in how efficiently they encode the decision boundary; the one with fewer support vectors generalises better.

Dataset

The file `dataset.npz` contains four arrays:

Key	Shape	Description
<code>X_train</code>	<code>(600, 2)</code>	training features
<code>y_train</code>	<code>(600,)</code>	training labels $\in \{0, 1\}$
<code>X_test</code>	<code>(200, 2)</code>	test features
<code>y_test</code>	<code>(200,)</code>	test labels $\in \{0, 1\}$

The two features are on different scales. Appropriate preprocessing is important for kernel methods.

Files provided

- `README.md`
- `dataset.npz`
- `student_svm.py`
- `evaluate.py` — local evaluation script

Files you may modify

- `student_svm.py`

Instructions

The starter file `student_svm.py` defines four functions. Implement each one according to its docstring. Do **not** change the function names or their signatures.

Part A - Data Loading and Preprocessing

- Load the `.npz` file.
- Fit a `StandardScaler` and then transform both training and test features.
- Return a dictionary with keys `"X_train"`, `"y_train"`, `"X_test"`, `"y_test"`, and `"scaler"` (the fitted `StandardScaler` instance).

Part B - Training

- Instantiate and fit a `sklearn.svm.SVC` with the given hyperparameters.
- Return the fitted model.

Part C - Evaluation

- Return a dictionary with keys containing the test accuracy, the number of support vectors used, and number of support vectors per class.

Part D - Optimal SVC

- Reason over SVC's so that they satisfy `accuracy >= accuracy_threshold` on the test set while minimising the total number of support vectors.
- Return a dictionary with keys containing all the specifications and hyperparameters of your model.

Public evaluator

The provided `evaluate.py` uses the following settings:

- time limit: `TIMEOUT = 60` seconds,
- accuracy threshold: `ACCURACY_THRESHOLD = 0.95`,
- support vector thresholds: `SV_EXCELLENT = 75`, `SV_GOOD = 100`, `SV_PARTIAL = 150`.

Run `python3 evaluate.py` to check your implementation locally.

Allowed libraries

Only `numpy`, `scikit-learn`, and the Python standard library may be used.

Submission instructions

Submit the single file:

- `student_svm.py`

Refer to Appendix for Q1: Notes on SVM Kernels and Regularisation (A)

Q2. Learning a Nonlinear Function with a FFNN [25 marks]

You are given starter NumPy code for a feed-forward neural network. The provided implementation is intentionally minimal and is only meant to serve as a starting point.

The target function in this problem is nonlinear and compositional. So the starter code, in its original form, is not expected to learn the target accurately within the evaluation budget. Your task is to modify the model and the training procedure so that it performs well on unseen inputs.

Task

Using the provided codebase, design and train a feed-forward neural network that approximates the target function accurately on unseen inputs from $[0, 1]$.

Target function

Define

$$\tau(x) = \max(0, 1 - 2|x - 0.5|), \quad x \in [0, 1].$$

The target function is

$$f(x) = \tau(\tau(\tau(\tau(x)))).$$

This function is available through `targets.py`.

Files provided

- `README.md`
- `targets.py`
- `student_model.py`
- `student_optimizer.py`
- `student_train.py`
- `evaluate.py`

Files you may modify

- `student_model.py`
- `student_optimizer.py`
- `student_train.py`

Instructions

- (1) Use **NumPy only**. Do not use PyTorch, TensorFlow, JAX, scikit-learn, or any other machine learning library.
- (2) The network input dimension must remain 1 and the output dimension must remain 1.
- (3) You may modify the architecture, forward pass, backward pass, optimizer, initialization, training loop, and data generation strategy.
- (4) Training data must be generated from the given target function over the interval $[0, 1]$.
- (5) The evaluator will instantiate `StudentClass` and call its method `fit_model(seed)`.
- (6) The trained model returned by `fit_model(seed)` must provide a method `predict(x)`.
- (7) Your code must run within the prescribed time budget on the evaluation machine and should achieve low mean squared error on unseen test inputs.

- (8) You may not use pretrained models, external datasets, or hard-coded outputs specific to the test inputs. (We will use **different functions** of the same form for testing, so hard-coding specific outputs will not work. The number of compositions will be of similar order, but the τ function can change)

Important interface note

Please do **not** change the signatures of the following methods:

- `StudentClass.build_model(self, seed=0)`
- `StudentClass.build_optimizer(self, name="adam")`
- `StudentClass.fit_model(self, seed=0)`

We may overwrite or monkey-patch `build_model` and `build_optimizer` during autograding. So even if you change the internals of these methods, their signatures must remain unchanged.

Public evaluator

The provided `evaluate.py` uses the following public settings:

- time limit: `TIMEOUT = 15.0` seconds,
- target accuracy: `TOLERANCE = 1e-2`,
- test set size: 2048 randomly sampled points from $[0, 1]$.

The public evaluator trains your model using

```
StudentClass().fit_model(seed=123)
```

and then checks the mean squared error on unseen test samples.

Autograding and partial credit

Your submission will be evaluated using the following three combinations:

- Your Optimizer + Our Model
- Our Optimizer + Your Model
- Your Optimizer + Your Model (If this works well, you will get the full credit)

A few important points:

- (1) The main goal is to obtain good performance with **your own model and your own optimizer**.
- (2) There will be partial credit for MSE higher than the threshold but zero credit for TIMEOUT violations. So it is important to ensure that your code runs within the time limit, even if it does not achieve the target accuracy.
- (3) The first two combinations are mainly for **partial credit** and to check whether your model and optimizer work reasonably well in isolation. If your model-optimizer pair works well, then we won't be checking these partial-credit combinations for the final grading.
- (4) To test these combinations, we may overwrite your `build_model` method or your `build_optimizer` method while keeping the rest of your code unchanged.

- (5) Since different model–optimizer pairs may have different running times, we may try a few epoch counts (typically multiples of 500) in some of the partial-credit evaluations, and use the best valid outcome among them.
- (6) This is only to make the grading fair. A slower model paired with a faster optimizer, or vice versa, should not be unfairly penalized just because the fixed epoch count was mismatched.

Remarks

The starter implementation is intentionally simple. You are expected to decide how to modify the network and training procedure so that the target can be learned reliably within the evaluation budget.

Also, do not overthink the starter code too much. A clean and sensible implementation is usually better than a very clever but fragile one.

Note: You are free to implement any optimizer. In practice, for this problem, adaptive optimizers such as RMSProp and Adam may perform better than plain gradient descent. **Refer to Appendix for Q2: Notes on Optimizers (B)**

Q3. Seeing What to Lose [30 marks]

This question asks you to *explain* a pretrained CNN and then *compress* it. You are given a CNN pretrained on the FashionMNIST dataset.

In Part 0, you start out with the concept of hooks in PyTorch, and do a simple exercise on the same.

In Part 1, you will implement Gradient-weighted Class Activation Mapping (Grad-CAM) and use it to compute a numerical importance score for every filter in the last convolutional layer.

In Part 2, you will use *any* importance criterion of your choice to prune the model down to a target size while retaining acceptable accuracy.

The last two parts share the same model but are otherwise independently graded.

Part-0: Feature Map Visualisation

Before implementing Grad-CAM, you will use a forward hook to inspect what the network has learned to detect. Concretely, for a single input image of your choice from `sample_images.pt`, register a forward hook on `block3[0]` to capture its output feature maps, then visualise all K maps as a grid. Feel free to refer to the appendix for a brief primer on hooks.

Write a function that returns the feature maps produced by `block3[0]` for a given input, without modifying the model or retaining any gradient information.

Part-1: Grad-CAM

Grad-CAM [1] produces a spatial heatmap that highlights the image regions most influential for a particular class prediction. Given a target convolutional layer ℓ with K filters, each producing an activation map of spatial size $H \times W$, let:

- $A^{k,\ell} \in \mathbb{R}^{H \times W}$ be the activation map of filter k at layer ℓ for a given input, ie, it is the output of the k -th filter at convolutional layer ℓ .
- y^c be the raw (pre-softmax) logit for class c .

Step 1: Filter importance weights The importance weight of filter k for class c is the global average pooled gradient of y^c with respect to A^k :

$$\alpha_k^{c,\ell} = \frac{1}{HW} \sum_{i=1}^H \sum_{j=1}^W \frac{\partial y^c}{\partial A_{ij}^{k,\ell}} \quad (1)$$

$\alpha_k^{c,\ell}$ is a scalar per filter; it measures how strongly filter k drives the score for class c .

Step 2: Weighted activation map The Grad-CAM heatmap at layer ℓ is the ReLU of the α -weighted sum of activation maps:

$$L_{\text{Grad-CAM}}^{c,\ell} = \text{ReLU} \left(\sum_{k=1}^{K_\ell} \alpha_k^{c,\ell} A^{k,\ell} \right) \in \mathbb{R}^{H_\ell \times W_\ell} \quad (2)$$

The ReLU retains only the spatial locations that *increase* y^c ; regions that suppress the predicted class are zeroed out.

Step 3: Upsample and normalise Let $H_{\text{in}} \times W_{\text{in}}$ be the spatial dimensions of the input image and $H \times W$ be the (smaller) spatial dimensions of $L_{\text{Grad-CAM}}^{c,l}$. Define the upsampled map $\tilde{L}^c \in \mathbb{R}^{H_{\text{in}} \times W_{\text{in}}}$ by applying bilinear interpolation to $L_{\text{Grad-CAM}}^{c,l}$, so that each entry $\tilde{L}_{ij}^c$ is a weighted average of the four nearest neighbours in $L_{\text{Grad-CAM}}^c$ under the mapping $(i, j) \mapsto (\frac{i \cdot H}{H_{\text{in}}}, \frac{j \cdot W}{W_{\text{in}}})$.

Normalise to $[0, 1]$ via

$$\hat{L}_{ij}^c = \frac{\tilde{L}_{ij}^c - \min_{i', j'} \tilde{L}_{i'j'}^c}{\max_{i', j'} \tilde{L}_{i'j'}^c - \min_{i', j'} \tilde{L}_{i'j'}^c} \quad (3)$$

If $\max_{i', j'} \tilde{L}_{i'j'}^c = \min_{i', j'} \tilde{L}_{i'j'}^c$ (i.e. the map is constant), set $\hat{L}^c = \mathbf{0}$.

The final heatmap is $\hat{L}^c \in [0, 1]^{H_{\text{in}} \times W_{\text{in}}}$.

Part-2: Filter Pruning

Structured pruning removes entire filters from a convolutional layer, producing a smaller but architecturally consistent model.

Let layer ℓ have weight tensor $W_\ell \in \mathbb{R}^{C_{\text{out}} \times C_{\text{in}} \times k_H \times k_W}$, followed by layer $\ell+1$ with weight tensor $W_{\ell+1} \in \mathbb{R}^{C'_{\text{out}} \times C_{\text{out}} \times k'_H \times k'_W}$. Removing filter i from layer ℓ requires four coordinated edits:

1. **Layer ℓ weights.** Delete index i along the C_{out} dimension of W_ℓ . Result: shape $(C_{\text{out}}-1) \times C_{\text{in}} \times k_H \times k_W$.
2. **Layer ℓ bias.** If $b_\ell \in \mathbb{R}^{C_{\text{out}}}$ exists, delete element i .
3. **Layer $\ell+1$ weights.** Filter i of layer ℓ produced the i -th *input channel* to layer $\ell+1$. Delete index i along the C_{in} dimension of $W_{\ell+1}$. Result: shape $C'_{\text{out}} \times (C_{\text{out}}-1) \times k'_H \times k'_W$.
4. **BatchNorm.** A `BatchNorm2d` following layer ℓ holds per-channel parameters $\gamma, \beta \in \mathbb{R}^{C_{\text{out}}}$ and running statistics $\mu, \sigma^2 \in \mathbb{R}^{C_{\text{out}}}$. Delete index i from each of these four vectors.

Construct a **new** `SmallCNN` and copy the pruned tensors into it; editing weights in-place is insufficient because the declared channel counts of the `nn.Conv2d` must also change.

Criteria for ranking filter importance

Any of the following heuristics may be used to decide which filters to remove (lowest score $\Rightarrow$ remove first):

- **ℓ_2 -norm:** $s_k = \|W_\ell[k]\|_2$. Filters with small norm contribute little to the output magnitude.
- **Variance:** $s_k = \text{Var}(W_\ell[k])$. Low-variance filters are near-constant and hence less expressive.
- **Grad-CAM importance** (Eq. (1)): $s_k = \bar{\alpha}_k$, the average α_k^c over a representative set of inputs and classes. This is a data-driven criterion that directly measures each filter's contribution to the model's predictions.

You are free to use any of the above, or a combination of the same.

Provided Materials

- `model.pth` - weights of a small CNN trained on a 10-class image classification task.
- `model.py` — the full model class definition. Read this carefully before starting.
- `test_loader.pkl` — a serialised `DataLoader` of 200 labelled test images.

- `sample_images.pt` — a tensor of shape $(10, C, H, W)$ containing one representative image per class, intended for use in Grad-CAM scoring.

Evaluation Criteria

Autograding note for Part 0. The output of your function will be graded against the correct output.

Autograding note for Part 1. Your `alpha_bar.pt` will be compared against a reference solution using `torch.allclose` with `atol=1e-3`.

Autograding note for Part 2. Your task is to prune the model by modifying it's last CNN layer, such that the following two constraints are satisfied:

1. Number of $< P_{max}$
2. Classification accuracy on Held out Dataset $> A_{min}$

We have given the `test_loader.py` as a small surrogate for the same.

The autograder will (a) load `pruned_model.pth` into the pruned architecture, run a forward pass to confirm it executes, and check `results['pruned_params'] ≤ Pmax`; and (b) re-evaluate the pruned model on a held-out test set and check that accuracy meets A_{min} . Marks are awarded only if *both* constraints are satisfied.

Submission instructions

Kindly submit the following files:

- `prune.py`
- `alpha_bar.pt`
- `pruned_model.pth`

All three files must be present for full autograding. Missing files score zero for the corresponding task.

Q4. Optimising with a Hidden Loss [30 marks]

You are given a one-dimensional regression dataset and a collection of *calibration samples* produced by an unknown loss function. The loss function and the ground truth labels for the calibration data is **not revealed** to you.

Your task is to *model* the hidden loss from the calibration data and using this loss, for the next stage, you build a model that optimises the predictions to minimise this unknown loss.

Note. The goal here is not to estimate y accurately in the least-squares sense, but to minimise the hidden loss. For example, if the loss penalises errors heavily near $x = 0$, your final linear predictor should prioritise accuracy in that region, even at the cost of larger errors elsewhere.

Background

A standard regression model minimises mean squared error, treating all input locations equally. In this problem the true loss is *not* squared error - it depends on the input location x , so errors in some regions are penalised more heavily than in others.

You are given calibration triplets $(x_i, y_{\text{pred},i}, \ell_i)$, each recording the loss ℓ_i incurred when prediction $y_{\text{pred},i}$ was made at location x_i . The true target y_{true} is **not provided** for the calibration points - you must infer it from the loss values.

The overall pipeline has four stages, each corresponding to one part of the question:

1. **Load** the data.
2. **Estimate** y_{true} from the calibration samples.
3. **Model** the loss surface $\ell(r, x)$ using a Nadaraya–Watson estimator.
4. **Fit** a linear predictor $\hat{y} = ax + b$ that minimises the learned loss.

Setup

The hidden loss function L depends on y_{true} , y_{pred} , and the input location x . Two properties are guaranteed:

1. L is minimised when $y_{\text{pred}} = y_{\text{true}}$.
2. The penalty for an error of the same magnitude may differ depending on x .

Dataset

The file `dataset.npz` contains:

Key	Shape	Description
<code>x_train</code>	$(n_{\text{train}},)$	training inputs
<code>y_train</code>	$(n_{\text{train}},)$	training targets
<code>x_test</code>	$(n_{\text{test}},)$	test inputs
<code>y_test</code>	$(n_{\text{test}},)$	test targets
<code>x_cal</code>	$(n_{\text{cal}},)$	calibration input locations
<code>y_pred_cal</code>	$(n_{\text{cal}},)$	calibration predictions
<code>loss_cal</code>	$(n_{\text{cal}},)$	observed loss values

Note that `y_true_cal` is **not provided**.

Files provided

- `README.md`
- `dataset.npz`
- `student_loss.py` - starter code with TODO stubs
- `evaluate.py` - local evaluation script

Files you may modify

- `student_loss_nw.py`

Instructions

The starter file `student_loss_nw.py` defines eight functions organised into four parts. Implement each one according to its docstring. Do **not** change the function names or their signatures.

Part A - Data Loading

Load the `.npz` file and return a dictionary containing every array listed in the dataset table above.

Part B - Target Estimation

The calibration data contains many $(x, y_{\text{pred}}, \ell)$ triplets but *not* the true target y_{true} . Since the loss is minimised when y_{pred} equals y_{true} , calibration samples with smaller loss values carry predictions that are *closer* to the truth.

Your task is to exploit this property to estimate y_{true} at an arbitrary set of evaluation locations `x_eval`.

Hint. Multiple calibration samples may share similar x values but differ in their predictions and losses. Grouping nearby calibration points, for instance by partitioning the x -axis into bins might help in giving a pointwise estimate of y_{true} . Interpolating between these estimates produces a smooth curve.

Part C - Nadaraya-Watson Estimator and Loss Modelling

With y_{true} estimated, you can compute the residual $r_i = |y_{\text{true},i}^{\text{est}} - y_{\text{pred},i}|$ for each calibration sample. The loss is now characterised by the triplets (r_i, x_i, ℓ_i) : the residual magnitude, the input location, and the observed loss.

Model the loss surface $\ell(r, x)$ using a **Nadaraya–Watson** (NW) kernel regression estimator, implemented **from scratch** using NumPy. The NW estimator predicts the loss at a new point (r, x) as a kernel-weighted average of the observed losses:

$$\hat{\ell}(r, x) = \frac{\sum_{i=1}^n K(\mathbf{z}, \mathbf{z}_i) \ell_i}{\sum_{i=1}^n K(\mathbf{z}, \mathbf{z}_i)}, \quad \mathbf{z} = (r, x),$$

where $K(\mathbf{z}, \mathbf{z}') = \exp(-\gamma \|\mathbf{z} - \mathbf{z}'\|^2)$ is the RBF kernel.

You will implement four functions for this part:

- `rbf_kernel` - compute the RBF kernel matrix between two sets of points.
- `nw_predict` - evaluate the NW weighted average given a kernel matrix.

- `model_loss` - prepare the NW model from the calibration residuals and losses (scale features, store training data).
- `predict_loss` - query the model at new (r, x) pairs.

See Appendix D for implementation guidance on the RBF kernel and feature scaling.

Part D - Linear Predictor Fitting

You now have a loss model that can be queried at *any* (r, x) pair. Use it to find the linear predictor $\hat{y} = ax + b$ that minimises the mean learned loss on the training data:

$$L_{\text{total}}(a, b) = \frac{1}{n} \sum_{j=1}^n \hat{\ell}(|y_j - (ax_j + b)|, x_j).$$

Return the fitted parameters a and b . You may use any optimisation strategy: grid search, gradient descent, or any other approach (see Appendix D for pointers on both).

Note. The goal is not to estimate y accurately in the least-squares sense, but to minimise the hidden loss. For example (This may not correspond to the given question), If the loss penalises errors heavily near $x = 0$, your predictor should prioritise accuracy in that region, even at the cost of larger errors elsewhere.

Public evaluator

- time limit: `TIMEOUT = 120` seconds.
- The evaluator reports whether your predictor beats OLS under the learned loss.

Run `python3 evaluate.py` to check your implementation locally.

Autograding and partial credit

Part	Criterion
A	Correct keys and shapes.
B	Mean absolute error of $\hat{y}_{\text{true}}$ against the true target function (tiered).
C	NW implementation correctness and loss-model quality on held-out data.
D	Mean hidden loss on held-out test data vs. OLS baseline (tiered).

Submission instructions

Submit the single file:

- `student_loss_nw.py`

Refer to Appendix for Q4: Notes on Nadaraya-Watson Regression (D)

A Notes on SVM Kernels and Regularisation for Q1

This appendix summarises the kernels available in `sklearn.svm.SVC` and the role of their main hyperparameters.

1. Linear kernel

$$K(\mathbf{x}, \mathbf{x}') = \langle \mathbf{x}, \mathbf{x}' \rangle.$$

2. Radial Basis Function (RBF) kernel

$$K(\mathbf{x}, \mathbf{x}') = \exp(-\gamma \|\mathbf{x} - \mathbf{x}'\|^2).$$

- $\gamma > 0$ controls the radius of influence of each training point.
- `gamma="scale"` sets $\gamma = 1/(d \cdot \text{Var}(X))$, where d is the number of features. `gamma="auto"` sets $\gamma = 1/d$.

3. Polynomial kernel

$$K(\mathbf{x}, \mathbf{x}') = (\gamma \langle \mathbf{x}, \mathbf{x}' \rangle + r)^d,$$

where d is the degree, γ is the coefficient, and r (`coef0`) is an independent term.

4. Useful **scikit-learn** attributes

After fitting an SVC object `clf`:

- `clf.n_support_` — array of support-vector counts per class.
- `clf.support_` — indices of support vectors in the training set.
- `clf.support_vectors_` — the support vectors themselves.
- `clf.score(X, y)` — classification accuracy on (X, y) .

B Note on Optimizers for Q2

Suppose a trainable parameter tensor is denoted by θ , and its gradient at the current step is denoted by

$$g_t = \nabla_{\theta} L,$$

where L is the loss.

1. Gradient descent

The simplest update rule is

$$\theta \leftarrow \theta - \eta g_t,$$

where $\eta > 0$ is the learning rate.

This uses the same learning rate for every coordinate of every parameter tensor. In deeper models, or when gradient magnitudes vary significantly across coordinates, such a rule may be difficult to tune.

2. RMSProp

RMSProp maintains an exponentially weighted moving average of squared gradients:

$$s_t = \beta s_{t-1} + (1 - \beta) g_t^2,$$

where:

- g_t^2 denotes elementwise squaring,
- β is typically chosen close to 1 (for example, 0.9),
- s_0 may be initialized to zero.

The parameter update is then

$$\theta \leftarrow \theta - \eta \frac{g_t}{\sqrt{s_t} + \varepsilon},$$

where ε is a small positive constant such as 10^{-8} .

Interpretation. Coordinates that consistently receive large gradients accumulate larger values in s_t , which reduces their effective step sizes. Coordinates with smaller gradients receive relatively larger updates. Thus RMSProp adapts the learning rate coordinate-wise.

Implementation note. For each weight matrix or bias vector, maintain an auxiliary NumPy array of the same shape to store s_t . Initialize it with zeros and update it at every optimization step.

3. Adam

Adam combines two moving averages:

$$\begin{aligned} m_t &= \beta_1 m_{t-1} + (1 - \beta_1) g_t, \\ v_t &= \beta_2 v_{t-1} + (1 - \beta_2) g_t^2, \end{aligned}$$

where:

- m_t is the first-moment estimate,

- v_t is the second-moment estimate,
- β_1 is commonly set to 0.9,
- β_2 is commonly set to 0.999.

Because both sequences start at zero, Adam applies bias correction:

$$\hat{m}_t = \frac{m_t}{1 - \beta_1^t}, \quad \hat{v}_t = \frac{v_t}{1 - \beta_2^t}.$$

The update rule is

$$\theta \leftarrow \theta - \eta \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \varepsilon}.$$

Interpretation. The first-moment term provides a smoothed descent direction, while the second-moment term rescales updates according to recent gradient magnitudes. Adam can therefore be viewed as combining momentum-like behavior with adaptive coordinate-wise step sizes.

Implementation note. For each trainable array, maintain:

- one array for m_t ,
- one array for v_t ,
- a scalar time-step counter t .

4. Typical hyperparameters

Reasonable defaults include:

- RMSProp: $\eta = 10^{-3}$, $\beta = 0.9$, $\varepsilon = 10^{-8}$,
- Adam: $\eta = 10^{-3}$, $\beta_1 = 0.9$, $\beta_2 = 0.999$, $\varepsilon = 10^{-8}$.

These are not mandatory. You are free to tune them.

5. Why adaptive methods may help

The target function is obtained by repeated composition of a nonlinear piecewise linear map. Such functions often induce optimization landscapes in which:

- different parameters experience gradients of very different magnitudes,
- some directions benefit from smaller effective step sizes than others,
- plain gradient descent may require more careful tuning.

Adaptive optimizers such as RMSProp and Adam automatically rescale updates coordinate-wise, which may improve stability within a limited time budget.

C Note on PyTorch Hooks for Q3

A *hook* is a callback function that PyTorch invokes automatically during the forward or backward pass of a module. Hooks let you capture intermediate activations and gradients without modifying the model definition.

Forward hooks fire after a module's `forward()` method completes:

```
def forward_hook(module, input, output):
    # 'output' is the tensor produced by this layer
    print(output.detach())

handle = model.conv2.register_forward_hook(forward_hook)
```

Backward hooks fire during the backward pass and expose the gradient of the loss with respect to that layer's output tensor:

```
def backward_hook(module, grad_input, grad_output):
    # 'grad_output[0]' is dL / d(output of this layer)
    print(grad_output[0].detach())

handle = model.conv2.register_full_backward_hook(backward_hook)
```

After you are done with a hook, always call `handle.remove()` to deregister it. Full documentation can be found in the given pytorch documentation.

D Notes on Nadaraya-Watson and Gradient Computation for Q5

1. Nadaraya-Watson estimator

Given training data $\{(\mathbf{z}_i, t_i)\}$ and an RBF kernel $K(\mathbf{z}, \mathbf{z}') = \exp(-\gamma\|\mathbf{z} - \mathbf{z}'\|^2)$, the NW prediction at a new point $\mathbf{z}$ is

$$\hat{f}(\mathbf{z}) = \frac{\sum_i K(\mathbf{z}, \mathbf{z}_i) t_i}{\sum_i K(\mathbf{z}, \mathbf{z}_i)} = \frac{S(\mathbf{z})}{T(\mathbf{z})}.$$

2. Pointers for gradient computation

Suppose we fit a linear model

$$y = f_\theta(x) = x^\top w + b,$$

where $w \in \mathbb{R}^d$ and $b \in \mathbb{R}$.

Our objective is to minimize the learned loss

$$\hat{\ell}(x, y) = \hat{\ell}(x, f_\theta(x)).$$

Chain Rule Decomposition

We compute gradients with respect to the parameters w and b using the chain rule:

$$\frac{\partial \hat{\ell}}{\partial w} = \frac{\partial \hat{\ell}}{\partial y} \cdot \frac{\partial y}{\partial w}, \quad \frac{\partial \hat{\ell}}{\partial b} = \frac{\partial \hat{\ell}}{\partial y} \cdot \frac{\partial y}{\partial b}.$$

Gradients of the Linear Model

From the definition $y = x^\top w + b$, we have:

$$\frac{\partial y}{\partial w} = x, \quad \frac{\partial y}{\partial b} = 1.$$

Final Gradient Expressions

Thus, the gradients become:

$$\frac{\partial \hat{\ell}}{\partial w} = \frac{\partial \hat{\ell}}{\partial y} x, \quad \frac{\partial \hat{\ell}}{\partial b} = \frac{\partial \hat{\ell}}{\partial y}.$$

References

- [1] Ramprasaath R Selvaraju, Michael Cogswell, Abhishek Das, Ramakrishna Vedantam, Devi Parikh, and Dhruv Batra. Grad-cam: Visual explanations from deep networks via gradient-based localization. In *Proceedings of the IEEE international conference on computer vision*, pages 618–626, 2017.